18. Listening Sockets, ss, and Firewalls Intro

18. Listening Sockets, ss, and Firewalls Intro

Phase: Phase 6 — Networking
Estimated time: 1–2 days
Slug: `18-ss-netstat-firewall`

Learning objectives

- List listening ports with ss
- Map ports to processes
- Explain firewall purpose (nftables/ufw)
- Open only needed ports

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

"Is my service actually listening?" is the first question in network debugging, and `ss` answers it in one line. The second question is "is something blocking it?", which brings in firewalls (`ufw`, `firewalld`, `nftables`, `iptables`). Together they separate a dead daemon from a blocked port, which saves hours.

`ss` replaced `netstat` on modern systems. It reads kernel socket tables directly, is faster, and shows more state. The classic flag combination is `-tulpn`, which you will type thousands of times in your career.

Core concepts

Flag	Meaning
`-t`	TCP sockets
`-u`	UDP sockets
`-l`	Listening sockets only
`-p`	Show owning process (needs root for others' processes)
`-n`	Numeric ports and addresses, no DNS lookups
`-a`	All states, not just listening
`-s`	Summary statistics

So `ss -tulpn` means: TCP and UDP, listening, with process, numeric.

Address columns matter:

Local address	Reachable from
`127.0.0.1:5432`	Only this machine
`0.0.0.0:22`	Any IPv4 interface
`[::]:22`	Any IPv6 interface (often IPv4 too)
`192.168.1.20:80`	Only that interface address

Packet path with a host firewall:

```
client --> NIC --> [ firewall rules ] --> socket (ss shows LISTEN) --> process
                          |
          drop/reject here: ss still shows LISTEN,
          but the client sees timeout (drop) or refused (reject)
```

Firewall front ends:

Tool	Distro family	Backend
`ufw`	Ubuntu/Debian	iptables/nftables
`firewalld` (`firewall-cmd`)	RHEL/Fedora	nftables
`nft`	all modern	kernel netfilter
`iptables`	legacy	kernel netfilter

Worked examples

List listeners:

```
sudo ss -tulpn
```

Sample output:

```
Netid State  Recv-Q Send-Q Local Address:Port Peer Address:Port Process
udp   UNCONN 0      0      127.0.0.53%lo:53   0.0.0.0:*  users:(("systemd-
resolve",pid=612,fd=13))
tcp   LISTEN 0      4096   0.0.0.0:22         0.0.0.0:*  users:
(("sshd",pid=801,fd=3))
tcp   LISTEN 0      511    127.0.0.1:8080     0.0.0.0:*  users:
(("python3",pid=2210,fd=4))
```

Who owns a given port:

```
sudo ss -tlpn 'sport = :22'
```

Sample output:

```
State  Recv-Q Send-Q Local Address:Port Peer Address:Port Process
LISTEN 0      4096   0.0.0.0:22         0.0.0.0:*  users:(("sshd",pid=801,fd=3))
```

Established connections and summary:

```
ss -tn state established | head -3
ss -s | head -2
```

Sample output:

```
Recv-Q Send-Q Local Address:Port   Peer Address:Port
0      0      192.168.1.20:22      192.168.1.5:50122
Total: 214
```

Firewall inspection and a minimal ufw policy:

```
sudo ufw status verbose
sudo ufw allow 22/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo nft list ruleset | head -5
```

Sample output:

```
Status: active
Default: deny (incoming), allow (outgoing)
22/tcp                     ALLOW IN    Anywhere
443/tcp                    ALLOW IN    Anywhere
```

The helper from the exercise:

```
ss_listening_flags() { echo "-tulpn"; }
sudo ss $(ss_listening_flags)
```

Mini-lab

1. Run `sudo ss -tulpn` and list every listening port with its process.
2. Start a throwaway listener: `python3 -m http.server 8088 &`.
3. Find it with `ss -tlpn | grep 8088`; note the local address.
4. Restart it bound to localhost only: `--bind 127.0.0.1`; compare columns.
5. Kill it with `kill %1`, confirm it vanished from `ss`.
6. On a VM, check the firewall: `sudo ufw status` or `sudo firewall-cmd --list-all`; write down which ports are open.
7. Allow a port, test it from another host with `nc -vz HOST 8088`, then remove the rule.
8. Run `ss -s` and compare totals before and after opening a few connections.

Decision guide / failure modes

Symptom	Meaning	Action
Port not in `ss -tulpn`	Daemon not running or wrong port	Check service logs
Listening on `127.0.0.1` only	Not reachable remotely	Bind `0.0.0.0` or proxy
Listening, remote times out	Firewall drops	Open port in firewall/security group
Listening, remote refused	Reject rule or wrong IP	Check rules and address
Process column empty	Not root	Use `sudo`
`Address already in use`	Another process owns port	`ss -tlpn 'sport = :PORT'`

Order of diagnosis: is it listening (ss), is it bound where expected, does the firewall allow it, and does the network path (cloud security group) allow it. Before enabling a firewall over SSH, always allow port 22 first, or you will lock yourself out.

Common mistakes

- Disabling firewall instead of opening one port
- Binding services to 0.0.0.0 unintentionally
- Forgetting IPv6 rules

Exercises

Work in `exercises/18-ss-netstat-firewall/`. Read that folder's `README.md` fully before coding.

```
cd exercises/18-ss-netstat-firewall
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement ss_listening_flags in exercise.sh. It takes no arguments and must print the exact flag string most admins use to list listening TCP and UDP sockets with owning process and numeric addresses. The test compares the output exactly, so print only the flags with the leading dash and no extra text. Before finishing, run sudo ss with those flags on a real host and write one sentence in your notes for each letter t, u, l, p and n, describing what it adds to the output.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/18-ss-netstat-firewall/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. ss -tulpn meaning?
2. Why default deny?
3. How allow SSH on ufw?

Further reading

- https://man7.org/linux/man-pages/man8/ss.8.html